



Chapter 18 Templates



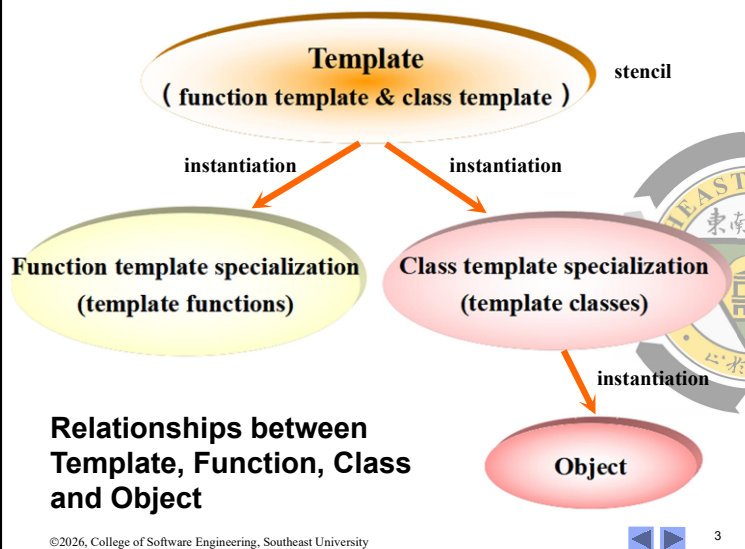
©2026, College of Software Engineering, Southeast University

18.1 Introduction

- Code Reuse: **Composite**, **Inherit**, **Template**
- Templates (模板)
 - Enable programmers to specify an entire range of related functions and related classes
 - Generic programming(泛型编程)
 - Parameterized type(参数化类型)
 - Template v.s. Template specialization(模板特化)
 - Function template, Class template

©2026, College of Software Engineering, Southeast University

2



©2026, College of Software Engineering, Southeast University

3

Topics

- 18.1 Introduction
- 18.2 Function Templates
- 18.3 Overloading Function Templates
- 18.4 Class Templates
- 18.5 Nontype Parameters and Default Types For Class Templates
- 18.6 Notes on Templates and Inheritance
- 18.7 Notes on Templates and Friends
- 18.8 Notes on Templates and static Members

©2026, College of Software Engineering, Southeast University

4

18.2 Function Templates(cont.)

- Function template definition
 - Preceded by a template header
 - Keyword **template**
 - Template parameter list (模板形参列表)
 - Enclosed in < and >
 - Each template parameter is preceded by keyword **class** or **typename**
 - Used to specify types of **parameters**, **local variables** and **return type** of function template
 - Examples


```
template< typename T >
template< class ElementType >
template< typename BorderType, class Filltype >
```

二者无差别

©2026, College of Software Engineering, Southeast University

5

18.2 Function Templates(cont.)

- Use of Function Template

```
template < typename T >  ← 模板形参T
T max ( T v1, T v2 ) { ..... }
```

— T 可以是任何基本数据类型或用户自定义类型。

— 函数模板只是对函数的说明，本身无法编译和执行，必须在编译时由编译器根据实际使用情况确认模板形参所对应的实参数据类型后，才能编译成相应的目标代码（特化）。

```
int main() {
    max(1, 0); // T is int      ← int为模板实参
    max(1.0, 0.0); // T is double ← double为模板实参
    string s1 = "hi", s2="world";
    max(s1, s2); // T is string ← string为模板实参
}
```

©2026, College of Software Engineering, Southeast University

6

```

1 // Fig. 18.1: fig18_01.cpp
2 #include <iostream> // cout, endl
3
4 template< typename T >
5 void printArray( const T *array, int count ) {
6     for ( int i = 0; i < count; i++ )
7         cout << array[ i ] << " ";
8     cout << endl;
9 }
10
11 int main()
12 {
13     const int ACOUNT = 5; // size of a
14     const int BCOUNT = 7; // size of b
15     const int CCOUNT = 6; // size of c
16
17     int a[ ACOUNT ] = { 1, 2, 3, 4, 5 };
18     double b[ BCOUNT ] = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6 };
19     char c[ CCOUNT ] = "HELLO"; // 6th position for null
20
21     cout << "Array a contains:" << endl;
22     printArray( a, ACOUNT );
23
24     cout << "Array b contains:" << endl;
25     printArray( b, BCOUNT );
26
27     cout << "Array c contains:" << endl;
28     printArray( c, CCOUNT );
29 }

```

Template parameter T specified in template header

```

void printArray( const int *array,
                 int count )
{
    for ( int i = 0; i < count; i++ )
        cout << array[ i ] << " ";
    cout << endl;
}

```

Creates a function-template specialization of printArray where double replaces T

Creates a function-template specialization of printArray where char replaces T

```

1 // Fig. 18.1: fig18_01.cpp
2 #include <iostream> // cout, endl
3
4 template< typename T >
5 void printArray( const T *array, int count ) {
6     for ( int i = 0; i < count; i++ )
7         cout << array[ i ] << " ";
8     cout << endl;
9 }
10
11 int main()
12 {
13     const int ACOUNT = 5; // size of a
14     const int BCOUNT = 7; // size of b
15     const int CCOUNT = 6; // size of c
16
17     int a[ ACOUNT ] = { 1, 2, 3, 4, 5 };
18     double b[ BCOUNT ] = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6 };
19     char c[ CCOUNT ] = "HELLO"; // 6th position for null
20
21     cout << "Array a contains:" << endl;
22     printArray( a, ACOUNT );
23
24     cout << "Array b contains:" << endl;
25     printArray( b, BCOUNT );
26
27     cout << "Array c contains:" << endl;
28     printArray( c, CCOUNT );
29 }

```

如果是用户自定义类型，必须要对流插入运算符<<进行重载！

注意：
如果模板被用户自定义类型调用，并且模板中对这些用户定义类型的对象使用了运算符（e.g., ==, +, <==），那么必须在该用户自定义类型中重载这些运算符。

```

GradeBook book[5];
printArray( book, 5 );

Complex c1, c2, c3;
c3 = max(c1, c2);

```

Topics

- 18.1 Introduction
- 18.2 Function Templates
- **18.3 Overloading Function Templates**
- 18.4 Class Templates
- 18.5 Nontype Parameters and Default Types For Class Templates
- 18.6 Notes on Templates and Inheritance
- 18.7 Notes on Templates and Friends
- 18.8 Notes on Templates and static Members

18.3 Overloading Function Templates

- Overload Function templates

– Between template functions:

```

printArray( int, ACOUNT );
printArray( double, ACOUNT );
printArray( char, ACOUNT );

```

– Between function templates :

Same function name, different parameter lists

```

template< typename T >
void printArray( const T *array, int count )

template< typename T >
void printArray( const T *array, int low, int high )

```

18.3 Overloading Function Templates(cont.)

- Overload Function templates(cont.)
- Between function templates and non-template functions:
Same function name, different parameter lists

```

template< typename T >
void printArray( const T *array, int count )

void printArray( const int *array, int count )

```

Topics

- 18.1 Introduction
- 18.2 Function Templates
- 18.3 Overloading Function Templates
- **18.4 Class Templates**
- 18.5 Nontype Parameters and Default Types For Class Templates
- 18.6 Notes on Templates and Inheritance
- 18.7 Notes on Templates and Friends
- 18.8 Notes on Templates and static Members

18.4 Class Templates

- **Class** is an abstract of problem space
- **Class template** is an abstract of classes
- A class template standards for a set of classes with the **same structure** but **different types**.

`vector<int> integers1(7);`
类模板, 属于C++ STL

18.4 Class Templates (cont.)

- Class Template Definition:

```
template < typename T >
class templateName
{
    // class definition body
};
```

- T is the type of class

- Type of data in member functions (i.e. parameters, local variables, return value)
- Type of data members

18.4 Class Templates (cont.)

- Class Template Definition:

- In class definition:

class template

泛型类名: `A<T>`

- In member function definition:

function template

在类模板定义体外定义成员函数时, 需使用泛型类名绑定类作用域。

```
template < typename T >
class A {
public:
    T getX();
    void setX( const T & );
private:
    T x;
};

template < typename T >
T A<T>:: getX() {
    return x;
}

template < typename T >
void A<T>:: setX( const T &a ){
    x = a;
}
```

18.4 Class Templates (cont.)

- Use of Class Template (Instantiation)

- 类模板只是对类的抽象 (parameterized type, 参数化类型), 它本身不是一个实实在在的类, 不能直接使用。

- 和使用普通类的区别在于: 类名需**指定类型参数**, 编译器依靠该信息对类模板进行特化, 生成相应的模板类 (template class)。

- Format: `templateName< type > obj;`

- Example:

18.4 Class Templates (cont.)

```
template < typename T >
class A {
public:
    T getX();
    void setX( const T & );
private:
    T x;
};

template < typename T >
T A<T>:: getX()
{
    return x;
}

template < typename T >
void A<T>:: setX( const T &a )
{
    x = a;
}
```

```
int main() {
    A< int > a;
    //生成int型的类A的源代码(模板类)

    a.setX(0);
    //调用类A<int>的成员函数模板特化:
    //void setX(const int &a)

    cout << a.getX();
    //调用类A<int>的成员函数模板特化:
    //int getX()

    A< double > b;
    b.setX(1.0);
    cout << b.getX();

    return 0;
}
```

```
1 // Fig. 18.2: Stack.h
2 // Stack class template.
3 #ifndef STACK_H
4 #define STACK_H
```

Create class template **Stack** with type parameter **T**

```
5
6 template< typename T >
7 class Stack
8 {
9 public:
10     Stack( int = 10 ); // default constructor (Stack size 10)
```

Member functions that use type parameter **T** in specifying function parameters

```
11
12 ~Stack()
13 {
14     delete [] stackPtr; // deallocate internal space for Stack
15 }
16
17 bool push( const T& ); // push an element onto the Stack
18 bool pop( T& ); // pop an element off the Stack
19
20 bool isEmpty() const
21 {
22     return top == -1;
23 }
24
25 }
```

```

27
28 bool isFull() const
29 {
30     return top == size - 1;
31 }
32
33 private:
34     int size;
35     int top;
36     T *stackPtr;
37 };
38
39 // constructor template
40 template< typename T >
41 Stack< T >::Stack( int s )
42 : size( s > 0 ? s : 10 ), // validate size
43   top( -1 ), // stack initially empty
44   stackPtr( new T[ size ] ) // allocate memory dynamically for elements
45 {
46     // empty body
47 }

```

Data member stackPtr is a pointer to a T array

Member functions, which are defined outside of class template definition, must start with their template headers

Notice: Using generic class name(泛型类名) Stack<T>

©2026, College of Software Engineering, Southeast University

```

49 // push element onto Stack;
50 template< typename T >
51 bool Stack< T >::push( const T &pushValue )
52 {
53     if ( !isFull() )
54     {
55         stackPtr[ ++top ] = pushValue; // first +1, then set item
56         return true;
57     }
58     return false;
59 }
60
61 // pop element off Stack;
62 template< typename T >
63 bool Stack< T >::pop( T &popValue )
64 {
65     if ( !isEmpty() )
66     {
67         popValue = stackPtr[ top-- ]; // first get item, then -1
68         return true;
69     }
70     return false; // if stack is empty, cannot pop
71 }
72
73
74
75 }

```

©2026, College of Software Engineering, Southeast University

```

1 // Fig. 18.3: fig18_03.cpp
2 // Stack class template test program.
3
4 #include "Stack.h" // Stack class template definition
5
6 int main()
7 {
8     Stack< double > doubleStack( 5 ); // double type Stack with size 5
9     double doubleValue = 1.1;
10
11     cout << "Pushing elements onto doubleStack\n";
12
13     while ( doubleStack.push( doubleValue ) ) // push until stack is full
14     {
15         cout << doubleValue << ' ';
16         doubleValue += 1.1;
17     }
18
19     cout << "\nStack is full. Cannot push " << doubleValue
20         << "\n\nPopping elements from doubleStack\n";
21
22     while ( doubleStack.pop( doubleValue ) ) // pop until stack is empty
23     {
24         cout << doubleValue << ' ';
25     }
26
27     cout << "\nStack is empty. Cannot pop\n";
28 }

```

©2026, College of Software Engineering, Southeast University

```

31
32 Stack< int > intStack; // default size 10
33 int intValue = 1;
34 cout << "\nPushing elements onto intStack\n";
35
36 // push 10 integers onto intStack
37 while ( intStack.push( intValue ) )
38 {
39     cout << intValue << ' ';
40     intValue++;
41 }
42
43 cout << "\nStack is full. Cannot push " << intValue
44     << "\n\nPopping elements from intStack\n";
45
46 // pop elements from intStack
47 while ( intStack.pop( intValue ) )
48     cout << intValue << ' ';
49
50 cout << "\nStack is empty. Cannot pop" << endl;
51 return 0;
52 }

```

©2026, College of Software Engineering, Southeast University

```

Pushing elements onto doubleStack
1.1 2.2 3.3 4.4 5.5
Stack is full. Cannot push 6.6

Popping elements from doubleStack
5.5 4.4 3.3 2.2 1.1
Stack is empty. Cannot pop

Pushing elements onto intStack
1 2 3 4 5 6 7 8 9 10
Stack is full. Cannot push 11

Popping elements from intStack
10 9 8 7 6 5 4 3 2 1
Stack is empty. Cannot pop

```

©2026, College of Software Engineering, Southeast University

```

1 // Fig. 18.4: fig18_04.cpp
2 // Stack class template test program. Function main uses a
3 // function template to manipulate objects of type Stack< T >.
4
5 #include "Stack.h" // Stack class template definition
6
7 // function template to manipulate Stack class-template specializations
8
9 template< typename T >
10 void testStack(
11     Stack< T > &theStack, // reference to Stack< T >
12     T value, // initial value to push
13     T increment, // increment for subsequent values
14     const string stackName ) // name of the Stack< T > object
15 {
16     cout << "\nPushing elements onto " << stackName << '\n';
17
18     // push element onto Stack
19     while ( theStack.push( value ) )
20     {
21         cout << value << ' ';
22         value += increment;
23     }
24 }

```

Use a function template to process Stack class-template specializations

©2026, College of Software Engineering, Southeast University

```

29
30 cout << "\nStack is full. Cannot push " << value
31     << "\n\nPopping elements from " << stackName << '\n';
32
33 // pop elements from Stack
34 while ( theStack.pop( value ) )
35     cout << value << ' ';
36
37 cout << "\nStack is empty. Cannot pop"
38 } // end function template testStack
39
40 int main()
41 {
42     Stack< double > doubleStack( 5 ); // size 5
43     Stack< int > intStack; // default size 10
44
45     testStack( doubleStack, 1.1, 1.1, "doubleStack" );
46     testStack( intStack, 1, 1, "intStack" );
47
48     return 0;
49 } // end main

```

The compiler infers type of **T** for **testStack** from the type of the function's **first** argument

©2026, College of Software Engineering, Southeast University

Topics

- 18.1 Introduction
- 18.2 Function Templates
- 18.3 Overloading Function Templates
- 18.4 Class Templates
- 18.5 Nontype Parameters and Default Types
For Class Templates
- 18.6 Notes on Templates and Inheritance
- 18.7 Notes on Templates and Friends
- 18.8 Notes on Templates and static Members

©2026, College of Software Engineering, Southeast University

18.5 Nontype Parameters and Default Types for Class Templates(cont.)

- Nontype template parameters(非类型模板形参)

– Template declaration:

```
template < typename T, int elements >
class Stack { ... };
```

– Template instantiation:

```
Stack< double, 100 > doubleStack;
```

- Value of **elements** can be confirmed at **compile time**.

```
const int size = 100;
Stack<int, size> s1; ✓
Stack<int, 100> s2; ✓
```

```
int n = 100;
Stack<int, n> s1; ✗
```

©2026, College of Software Engineering, Southeast University

18.5 Nontype Parameters and Default Types for Class Templates(cont.)

```

template< typename T, int elements = 5 >
class Stack {
public:
    Stack();
    bool push( const T& );
    bool pop( T& );
    bool isEmpty() const { return top == -1; }
    bool isFull() const { return top == elements- 1; }
private:
    int top;
    T stackPtr[elements];
};

```

How to use ?

Member **size** is not necessary

©2026, College of Software Engineering, Southeast University

18.5 Nontype Parameters and Default Types for Class Templates(cont.)

```

template< typename T, int elements >
Stack< T, elements >::Stack() : top( -1 ) // 只需对top初始化
{
    for (int i=0; i<elements; i++)
        stackPtr[i] = 0;
}

template< typename T, int elements >
bool Stack< T, elements >::push( const T &pushValue )
{
    if ( !isFull() ) {
        stackPtr[ ++top ] = pushValue;
        return true;
    }
    return false;
}

```

©2026, College of Software Engineering, Southeast University

18.5 Nontype Parameters and Default Types for Class Templates(cont.)

```

template< typename T, int elements >
bool Stack< T, elements >::pop( T &popValue )
{
    if ( !isEmpty() ) {
        popValue = stackPtr[ top-- ];
        return true;
    }
    return false;
}

Stack<int, 100> intStack; // Stack<int> intStack; is also ok
intStack.push(1);
intStack.push(2);

Stack<double, 200> doubleStack;
doubleStack.push(1.0);
doubleStack.push(2.0);

```

©2026, College of Software Engineering, Southeast University

18.5 Nontype Parameters and Default Types for Class Templates(cont.)

- Explicit specializations(显式模板特化)
 - Used when a particular type will not work with the general template or requires customized processing
 - Format:


```
template<>
class Stack< Employee > { ... };
```
 - Are complete replacements for the general template
 - Can even have different members
 - Example:

©2026, College of Software Engineering, Southeast University

31

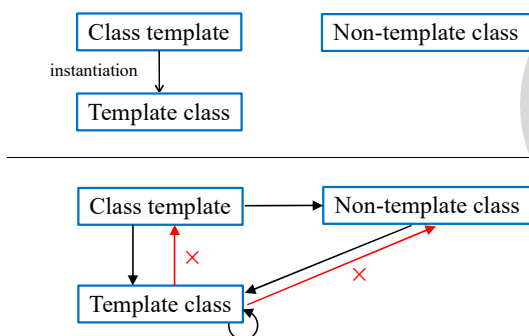
Topics

- 18.1 Introduction
- 18.2 Function Templates
- 18.3 Overloading Function Templates
- 18.4 Class Templates
- 18.5 Nontype Parameters and Default Types For Class Templates
- 18.6 Notes on Templates and Inheritance
- 18.7 Notes on Templates and Friends
- 18.8 Notes on Templates and static Members

©2026, College of Software Engineering, Southeast University

18.6 Notes on Templates and Inheritance

- Inheritance relationship



©2026, College of Software Engineering, Southeast University

33

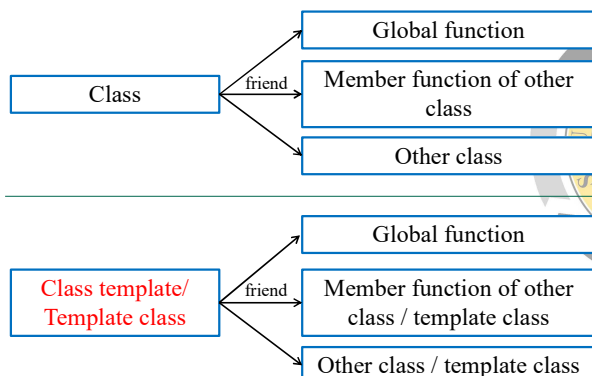
Topics

- 18.1 Introduction
- 18.2 Function Templates
- 18.3 Overloading Function Templates
- 18.4 Class Templates
- 18.5 Nontype Parameters and Default Types For Class Templates
- 18.6 Notes on Templates and Inheritance
- 18.7 Notes on Templates and Friends
- 18.8 Notes on Templates and static Members

©2026, College of Software Engineering, Southeast University

18.7 Notes on Templates and Friends

- friend relationship



©2026, College of Software Engineering, Southeast University

35

18.7 Notes on Templates and Friends (cont.)

- A member function of another class can be the friend of every template class instantiated from a class template
 - `friend void A::f3();`
 - `f3` of class `A` is a friend of `X< double >`, `X< string >`, etc.
- A member function of another class template can be the friend of only a template with the same type argument
 - `friend void C< T >::f4( X< T > & );`
 - `C< float >::f4( X< float > & )` is a friend of `X< float >`, but not a friend of `X< string >`

©2026, College of Software Engineering, Southeast University

36

18.7 Notes on Templates and Friends (cont.)

- Another class can be the friend of every template class instantiated from a class template
 - `friend class Y;`
 - Every member function of class Y is a friend of `X< double >`, `X< string >`, etc.
- A class template can be the friend of only a template class with the same type argument
 - `friend class Z< T >;`
 - `Z< float >` is a friend of `X< float >`
 - `Z< string >` is a friend of `X< string >`

1: N

1: 1

Topics

- 18.1 Introduction
- 18.2 Function Templates
- 18.3 Overloading Function Templates
- 18.4 Class Templates
- 18.5 Nontype Parameters and Default Types For Class Templates
- 18.6 Notes on Templates and Inheritance
- 18.7 Notes on Templates and Friends
- 18.8 Notes on Templates and static Members

18.8 Notes on Templates and static Members

- **static** data members of a class template
 - Each class-template specialization has its own copy of each **static** data member
 - All objects of that specialization share that one **static** data member
 - **static** data members must be defined and, if necessary, initialized at file scope
 - Each class-template specialization gets its own copy of the class template's **static** member functions

Ex1

```
#include <iostream>
using namespace std;

template <typename T>
class A
{
public:
    static int x;
};

template <typename T>
int A<T>::x = 0;

int main()
{
    A<int>::x = 10;
    A<double>::x = 20;
    A<int> a1, a2; // a1和a2共享一个x
    cout << a1.x << " ";
    a1.x = 15;
    cout << a2.x << endl;
    A<double> b1, b2; // b1和b2共享一个x
    cout << b1.x << " " << b2.x << endl;
    return 0;
}
```

10	15
20	20

Template v.s. Inheritance

- When the type of object doesn't affect the definition of member function of class, we use **class template** to generate this group of classes.
当对象的类型不影响类中函数的行为时，就要使用类模板来生成这样一组类。
- When the type of object affect the definition of member function of class, we use **inheritance** to generate this group of classes.
当对象的类型影响类中函数的行为时，就要使用继承来得到这样一组类。